

LZ78 Compression and Decompression Report

1. Introduction

The purpose of this report is to analyze the implementation of the LZ78 compression and decompression algorithm. LZ78 is a lossless data compression technique that builds a dictionary of unique strings encountered in the input text. This implementation provides an interactive console program that allows users to input text for compression and decompression using LZ78.

2. Algorithm Overview

LZ78 operates by scanning the input string and building a dictionary. As it reads the input, it tries to find the longest prefix that already exists in the dictionary. The match is then encoded as a tuple (index, next character), where:

- **Index** represents the dictionary index of the longest matching prefix.
- **Next Character** is the character that follows the matched prefix in the input.

The decompression process reconstructs the original string by using these encoded tuples to retrieve matching sequences from the dictionary, which is built during compression.

3. Code Implementation

The implementation consists of the following components:

- **Tag Class:** Represents the compression tuples containing the dictionary index and the next character.

- **Compression Method:**
 - Iterates through the input string, building a dictionary of encountered string prefixes.
 - Encodes each prefix and the following character as a tag and adds it to the compressed output.
 - Writes the binary representation of the tags to a .bin file, including the number of bits used for the index.
- **Decompression Method:**
 - Reads the encoded tags from a .bin file and reconstructs the original string by referencing entries in the dictionary.
 - Rebuilds the dictionary during the decompression process.
 - Writes the decompressed text to a .txt file.
- **User Interface:**
 - Provides options for compression, decompression, and exiting the program.
 - Reads and writes data from and to files.
- **Binary File Handling:**
 - Handles the conversion of tags into binary data and back, including padding and the storage of the index bit length.

4. Handling of Repetitive Sequences

The program effectively handles repetitive sequences by building a dictionary of previously encountered patterns. This allows it to efficiently encode subsequent occurrences of these patterns. The dynamic dictionary ensures that the compression adapts to the input data, making it suitable for various types of

repetitive text. The implementation ensures that the decompressed output is identical to the original input.

5. Example Test Cases

Compression Example:

- **Input (input.txt):**
 - ABABCBABAAAAAA
- **Output (compressed.bin):**
 - The binary file will contain the tags encoded as binary data, with the first byte indicating the number of bits used for the index.
 - The binary representation of the tags:
(0,A),(0,B),(2,C),(2,B),(1,A),(5,A),(1,A)
- **Size Calculation:**
 - Original text size: 14 bytes (112 bits).
 - Compressed data size: Calculated based on the number of tags and the bits needed for the index, plus the first byte that stores the bit length. The index uses 3 bits, and the character uses 8 bits. Thus 11 bits per tag, plus the initial 8 bits. $11 \times 7 + 8 = 85$ bits. $85 \text{ bits} / 8 = 10.625$, therefore 11 bytes.

Decompression Example:

- **Input (compressed.bin):**
 - The binary data from the previous compression example.
- **Output (decompressed.txt):**
 - ABABCBABAAAAAA

Compression Example 2:

- **Input (input2.txt):**
- HelloHelloHello
- **Output (compressed.bin):**
 - The binary representation of the tags:
(0,H),(0,e),(0,l),(3,o),(1,l),(2,o),(5,H),(0,e),(4,l),(6,l),(4,o)
- **Size Calculation:**
 - Original text size: 13 bytes (104 bits).
 - Compressed data size: The index uses 4 bits, and the character uses 8 bits. Thus 12 bits per tag, plus the initial 8 bits. $12 \times 11 + 8 = 140$ bits. $140 \text{ bits} / 8 = 17.5$, therefore 18 bytes.

Decompression Example 2:

- **Input (compressed.bin):**
 - The binary data from the previous compression example.
- **Output (decompressed.txt):**
- HelloHelloHello

6. Conclusion

This LZ78 implementation successfully compresses and decompresses text while preserving data integrity. It effectively builds and utilizes a dictionary to encode repetitive sequences, making it suitable for lossless text compression applications. Future improvements could include optimizing dictionary management and improving the efficiency of binary data handling.